Operatoren

Joern Ploennigs · AI4SC



> Clarity in thought, like elegance in mathematics, arises not from abundance but from precision.
>
> — Blaise Pascal

Anweisungen

Bei der Programmierung schreibt man Code, der formal aus Syntax und Semantik besteht. Dieser Code wird weiter unterteilt in:

Anweisung

Eine Anweisung (engl. statement) stellt eine in der Syntax einer Programmiersprache formulierte einzelne Vorschrift dar, die im Programm nacheinander auszuführen ist.

Ausdrücke

Eine besondere Form von Anweisungen sind Ausdrücke (engl. expression), die immer einen Wert zurückgeben und damit einen Datentyp haben.

Operator

Ein Operator ist ein Teil von Ausdrücken und ein Symbol oder Schlüsselwort, das dem Compiler oder Interpreter mitteilt, eine bestimmte Operation auf Werte auszuführen.

Bei einer einfachen Variablenzuweisung steht der Ausdruck zum Beispiel auf der rechten Seite des Zuweisungsoperators und der Variablenname auf der linken Seite:

```
Variablenname = Ausdruck
```

In dieser Sitzung geht es darum, solche Ausdrücke zu lesen und nachzuverfolgen, die ein KI-Assistent erzeugt hat, statt sie von Grund auf selbst zu schreiben.

Arithmetische Operatoren

Arithmetische Operatoren führen die bekannten mathematischen Operationen aus. Sie arbeiten mit numerischen Datentypen und werden mit numerischen Werten verwendet, um allgemeine mathematische Operationen auszuführen.

Operator	Beispiel	Wirkung
Addition	`x + y`	`10 + 3` (Ergebnis: 13)
Subtraktion	`x - y`	`10 - 3` (Ergebnis: 7)
Multiplikation	`x * y`	`10 * 3` (Ergebnis: 30)
Division	`x / y`	`10 / 3` (Ergebnis: 3.3333…)
Ganzzahl-Division	`x // y`	`10 // 3` (Ergebnis: 3)
Modulo-Operator	`x % y`	`10 % 3` (Ergebnis: 1)
Potenzierung	`x ** y`	`10 ** 3` (Ergebnis: 1000)

Bei Divisionen lohnt sich ein genauer Blick auf den Unterschied zwischen `/` und `//`: `7 / 3` ergibt `2.333...`, während `7 // 3` zu `2` abrundet. Bei negativen Zahlen wird Richtung minus unendlich gerundet, daher ist `-7 // 3` gleich `-3` und nicht `-2`.

Der Ausdruck `x ** 2 - 3 * x + 1` kombiniert Potenzierung, Multiplikation, Subtraktion und Addition. Gerade bei KI-generiertem Code ist es wichtig, zuerst die Operatorrangfolge zu erkennen und danach die Zwischenschritte nacheinander auszuwerten: Für `x = 5` ergibt sich `25 - 15 + 1 = 11`.

Zuweisungsoperatoren

Zuweisungsoperatoren weisen Variablen Werte zu. Python verwendet für alle arithmetischen Operatoren die abgekürzte Zuweisung, es gibt jedoch keine Inkrementierung oder Dekrementierung wie in anderen Sprachen.

Bei der Zuweisung `x = y` wird der Wert der Variablen `y` (rechte Seite) der Variablen `x` (linke Seite) zugewiesen. Bei abgekürzten Zuweisungen wird jeweils eine Operation auf dem Wert einer Variablen ausgeführt und dieser neue Wert erneut der Variablen zugewiesen.

Operator	Beispiel	Wirkung
Zuweisung	`x = y`	Variable `x` erhält den Wert `y`
Abkürzende Addition	`x += 4`	`x = x + 4` # Variablen `x` wird um 4 erhöht
Abkürzende Multiplikation	`x *= 4`	`x = x * 4` # Variablen `x` wird mit 4 multipliziert

Operator	Beispiel	Wirkung
Abkürzende Division	`x /= 4`	`x = x / 4` # Variablen x wird durch 4 dividiert
Abkürzende Ganzzahl-Division	`x //= 4`	`x = x // 4` # Variablen x wird durch 4 ohne Rest dividiert
Abkürzende Modulo	`x %= 4`	`x = x % 4` # Der Rest der Division von Variablen x durch 4

Anwendung von Zuweisungs- und arithmetischen Operatoren

Operatoren werden in Programmiersprachen verwendet, um zwei oder mehrere Werte oder Variablen miteinander zu verknüpfen. Die arithmetischen Operatoren `+`, `-`, `*`, `/` aus der Mathematik gibt es auch in Python.

> **i** KI-generierter Code
>
> Von KI generiert: erst nachverfolgen, dann ausführen.

```
a = 1 + 1
a  # 2
```

```
2
```

Die Operationen funktionieren genauso mit Variablen oder gemischt.

> **i** KI-generierter Code
>
> Von KI generiert: erst nachverfolgen, dann ausführen.

```
a = a + 1
a  # 3
```

```
3
```

In diesem Beispiel ist die erste Eingangsvariable `a` identisch mit der Ergebnisvariablen. In diesem Fall bietet Python auch eine Kurzform an, bei der der Operator direkt vor dem Zuweisungsoperator = steht.

```
a += 1
a  # 4
```

```
4
```

```
a *= 2
a  # 8
```

```
8
```

Hierbei bestimmen Klammern auch die Berechnungsreihenfolge, genauso wie in der Mathematik, und können das Ergebnis beeinflussen. Daher sind die folgenden beiden Ausdrücke nicht identisch:

```
c1 = (6 + a) * (-2)
c1  # -28
```

```
-28
```

```
c2 = 6 + a * -2
c2  # -10
```

```
-10
```

Vergleichsoperatoren

Vergleichsoperatoren vergleichen zwei Werte und liefern wahr oder falsch als Ergebnis. Zum Beispiel testet `a == b` ob `a` gleich `b` ist, und liefert je nachdem wahr oder falsch zurück, und `a != b` testet, ob `a` ungleich `b` ist.

Bei der Zuweisung `x = y` wird der Wert der Variablen `y` (rechte Seite) der Variablen `x` (linke Seite) zugewiesen. Vergleichsoperatoren sehen ähnlich aus, verändern aber keine Variable, sondern liefern nur einen Wahrheitswert.

Operator	Beispiel	Wirkung
Gleichheitsvergleich	`x == y`	wahr, wenn x gleich y ist
Ungleich	`x != y`	wahr, wenn x ungleich y ist
Größer	`x > y`	wahr, wenn x größer als y ist

Operator	Beispiel	Wirkung
Kleiner	`x < y`	wahr, wenn x kleiner als y ist
Größer gleich	`x >= y`	wahr, wenn `x` größer gleich `y` ist
Kleiner gleich	`x <= y`	wahr, wenn `x` kleiner gleich `y` ist

Identitätsoperatoren

Identitätsoperatoren vergleichen Objekte. Sie prüfen, ob zwei Variablen auf dasselbe Objekt im Speicher zeigen. `x is y` gibt wahr zurück, wenn beide Variablen dasselbe Objekt referenzieren.

Operator	Beispiel	Wirkung
Identität	`x is y`	wahr, wenn beide Variablen dasselbe Objekt sind.
Nicht gleich	`x is not y`	wahr, wenn die Variablen nicht dasselbe Objekt sind.

Ein gutes Gegenbeispiel ist eine Liste mit gleichem Inhalt: `a = [1, 2, 3]`, `b = a` und `c = [1, 2, 3]`. Dann ist `a is b` wahr, aber `a is c` falsch, obwohl `a == c` wahr ist.

Logische Operatoren

Logische Operatoren kombinieren boolesche Ausdrücke, die dann als wahr oder falsch ausgewertet werden. Z.B. wird hier die Variable `is_teenager = (alter > 13) and (alter < 19)` als wahr ausgewertet, wenn der Wert der Variablen alter gleich 14 ist.

`P` und `Q` stehen für logische Ausdrücke, z.B. `P = (x > 10)` und `Q = (x < 20)`.

Operator	Beispiel	Wirkung
Logisches Und	`P and Q`	Gibt wahr (1) zurück, wenn die Ausdrücke `P` und `Q` beide wahr sind
Logisches Oder	`P or Q`	Gibt wahr (1) zurück, wenn einer der Ausdrücke `P` oder `Q` wahr ist
Logische Verneinung	`not P`	Verneint den Ausdruck `P`: aus wahr wird falsch, aus falsch wird wahr

Anwendung von Vergleichs- und Logischen Operatoren

Um zu prüfen, ob der Wert zweier Variablen gleich ist, wird in den meisten Programmiersprachen nicht das Gleichheitszeichen = verwendet, das es dort meist als Zuweisungsoperator definiert ist (und somit die erste Variable überschreiben würde). In Python nutzt man deshalb ein doppeltes Gleichheitszeichen == (mathematisch für exakt das gleiche). In unserem Beispiel ist `c1` nicht gleich `c2`, also die folgende Aussage falsch.

```
c1 == c2
```

```
False
```

Für Ungleichheit wird in Python der `!=` Operator verwendet

```
c1 != c2
```

```
True
```

Auch die mathematischen Operatoren für kleiner <, kleiner gleich <=, größer >, und größer gleich >= werden unterstützt.

```
c1 < c2
```

```
True
```

```
c1 <= c2
```

```
True
```

```
c1 > c2
```

```
False
```

```
c1 >= c2
```

```
False
```

Boolesche (logische) Werte werden in Python nicht durch die üblichen Operatoren für UND &, ODER |, NOT ~ kombiniert, sondern als `and`, `or`, `not` ausgeschrieben. Das Größergleich >= ist z.B. identisch mit der Bedingung

```
c1 > c2 or c1 == c2
```

```
False
```

Operatorrangfolge

Python wertet Operatoren nicht von links nach rechts aus, sondern nach ihrer Rangfolge: Potenzierung vor Punktrechnung, Punkt- vor Strichrechnung, arithmetische Operatoren vor Vergleichen, Vergleiche vor `not`, `not` vor `and`, `and` vor `or`. Bei KI-generierten Ausdrücken lohnt es sich, diese Reihenfolge vor dem Ausführen nachzuvollziehen. Klammern machen die gewünschte Reihenfolge explizit und sind im Zweifel besser als Wissen über die Rangfolge.

Agentischer Arbeitsablauf: Logische Ausdrücke

Bei KI-generierten Bedingungen lohnt sich eine kurze Lesepause vor dem Ausführen. Ein Ausdruck wie `if not x is None and x > 0.0:` wird leichter verständlich, wenn man erst die Rangfolge prüft: Die Vergleiche `x is None` und `x > 0.0` werden zuerst ausgewertet, danach wird `not` auf `x is None` angewendet und zuletzt beide Teile mit `and` verknüpft. Man überführt ihn danach in die klarere Form `x is not None and x > 0.0`.

Bitweise Operatoren

Bitweise Operatoren führen bitweise Operationen auf ganzzahligen Operanden aus. D.h. die Operanden werden in ihre bitweise Darstellung konvertiert, z.B. `x = 5 = 0101`, `y = 3 = 0011`, und dann wird jedes Bit im Operanden `x` mit dem Bit an der passenden Position im Operanden `y` verknüpft.

Operator	Beispiel	Wirkung
Bitweises Und	`x & y`	Ergebnisbit ist 1, wenn beide Bits 1 sind.
Bitweises Oder	`x | y`	Ergebnisbit ist 1, wenn mindestens eines der Bits 1 ist.
Bitweises Entweder-Oder (XOR)	`x ^ y`	Ergebnisbit ist 1, wenn genau eines der Bits 1 ist.
Bitweises Nicht	`~x`	Gibt das Einerkomplement von `x` zurück.
Shift-right	`x >> n`	Bits in `x` werden um `n` Positionen nach rechts geschoben.

Operator	Beispiel	Wirkung
Shift-left	`x << n`	Bits in `x` werden um `n` Positionen nach links geschoben.

Die üblichen logischen Operatoren UND &, ODER |, sowie XOR ^ sind für bitweise Verknüpfung reserviert. Dies benutzt man nur sehr selten, bei bestimmten Berechnungen oder beim Masking.

```
a = 10 # binär 1010
b = 4  # binär 0100
```

```
c = a & b
print(c) # binär 0000
```

```
0
```

```
c = a | b
print(c) # binär 1110
```

```
14
```

Mit den Linksverschiebungs- << und Rechtsverschiebungs-Operatoren >> lassen sich auch sehr schnelle Berechnungen realisieren. Eine Multiplikation mit 4 ist zum Beispiel eine Linksverschiebung um 2, eine Division um 4 hingegen eine Rechtsverschiebung um 2. Dies nutzen Compiler intern, um Berechnungen zu optimieren.

```
c = a << 2
print(c)
```

```
40
```

```
d = c >> 2
print(d)
```

```
10
```

Überladen von Operatoren

Die arithmetischen Operatoren + und - sind in Python *überladen*, d.h. man kann sie auch auf nicht numerische Datentypen anwenden. Um mehrere Strings zu verbinden kann man zum Beispiel schreiben:

> **i** KI-generierter Code — Datentypen prüfen
>
> Von KI generiert: erst nachverfolgen, dann ausführen.

```
satz = "Der "+ "Ball " + "ist " + "rund."
print(satz)
```

```
Der Ball ist rund.
```

Dies funktioniert auch für Listen

```
liste1 = [1,2,3]
liste2 = [2,3,4]

liste12 = liste1 + liste2
print(liste12)
```

```
[1, 2, 3, 2, 3, 4]
```

oder als Subtraktion für Mengen

```
menge1 = {1,2,3}
menge2 = {2,3,4}

menge12 = menge1 - menge2
print(menge12)
```

```
{1}
```

⚠ Achtung

Hierbei ist zu beachten, dass die Operatoren ihre semantische Bedeutung ändern, je nachdem welche Datentypen kombiniert werden. So ist die Addition zweier Strings `str` formal eine `concat` in einen neuen String. Bei der Liste bedeutet + das Kopieren der ersten Liste in eine neue, welche dann mit `expand` um den Inhalt der nächsten Liste erweitert wird. Bei beiden ist der Operator für die Subtraktion - nicht definiert. Bei einem Set ist + nicht definiert, dafür aber - um Differenzmenge zu bestimmen.

Ferner müssen bei Python beide verknüpften Werte immer den gleichen Datentypen haben. So kann man nicht, wie in Java oder JavaScript, ein String und eine Nummer kombinieren, sondern muss die Nummer zuerst in einen String konvertieren (casting).

ℹ KI-generierter Code — häufiger Typfehler

Von KI generiert: erst nachverfolgen, dann ausführen.

```
"Der Wert ist " + 1  # geht nicht
```

```
TypeError: can only concatenate str (not "int") to str
?[31m---------------------------------------------------------------------------?
[39m
?[31mTypeError?[39m                                 Traceback (most recent
call last)
?[36mCell?[39m?[36m ?[39m?[32mIn[22]?[39m?[32m, line 1?[39m
?[32m----> ?[39m?[32m1?[39m ?[33;43m"?[39;49m?[33;43mDer Wert ist
?[39;49m?[33;43m"?[39;49m?[43m ?[49m?[43m+?[49m?[43m
?[49m?[32;43m1?[39;49m  ?[38;5;66;03m# geht nicht?[39;00m

?[31mTypeError?[39m: can only concatenate str (not "int") to str
```

```
"Der Wert ist " + str(1)  # geht
```

```
'Der Wert ist 1'
```

Dies hat damit zu tun, dass bestimmte Statements sonst mehrdeutig wären. So ist hier unklar, ob das Ergebnis ein String sein soll oder eine Zahl.

```
"1" + 1  # geht nicht
```

```
TypeError: can only concatenate str (not "int") to str
[31m---------------------------------------------------------------------------[39m
[31mTypeError[39m                                 Traceback (most recent call last)
[36mCell[39m[36m [39m[32mIn[24][39m[32m, line 1[39m
[32m----> [39m[32m1[39m
[33;43m"[39;49m[33;43m1[39;49m[33;43m"[39;49m[43m
[49m[43m+[49m[43m [49m[32;43m1[39;49m  [38;5;66;03m# geht nicht[39;00m

[31mTypeError[39m: can only concatenate str (not "int") to str
```

Durch Casting der Werte in den richtigen Typ wird das Statement eindeutig

```
int("1") + 1  # geht
```

```
2
```

Mitgliedsoperatoren

Mitgliedsoperatoren prüfen, ob ein Objekt Mitglied einer Liste ist. Z.B. prüft `4 in [1,2,3]`, ob 4 in der Liste enthalten ist.

`x` steht hier für ein beliebiges Objekt, `list` für eine Auflistung.

Operator	Beispiel	Wirkung
enthalten in	`x in list`	wahr, wenn x in der Liste `list` enthalten ist.
nicht enthalten	`x not in list`	wahr, wenn x nicht in der Liste `list` enthalten ist.

Tips zur Anwendung

Mathematische Operatoren haben auch eine Kurzversion die vor dem Zuweisungsoperator `=` stehen kann: z.B. `a += 1` entspricht `a = a + 1`, `a *= 2` ist äquivalent zu `a = a*2`.

```
i = 1
i *= 2
i *= 3
i  # 6
```

```
6
```

Logische Operationen funktionieren meistens wie erwartet. Insbesondere >, <, >= und <=. Wie in mehreren anderen Sprachen wird Gleichheit mit doppelten Gleichheitszeichen == getestet. Ungleichheit kann mit != getestet werden, und die logische Negation ist `not`:

```
a = 1
a > 1  # False
a >= 1  # True
a == 1  # True
a != 1  # False
not a == 1  # False
```

```
False
```

Python unterstützt auch etwas weniger verbreitete, aber äußerst praktische Mehrweg-Vergleichsoperationen, zum Beispiel

```
0 < a < 2  # True
```

```
True
```

Python erlaubt solche Mehrfachvergleiche, ohne dass `a` doppelt geschrieben werden muss. Das ist oft lesbarer als `0 < a and a < 2`.

Übungsaufgabe: KI-Code nachverfolgen

Auch hier hilft die Strategie aus der ganzen Sitzung: Operatoren identifizieren, Rangfolge bestimmen und dann den Ausdruck schrittweise auswerten. Bei `not x is None and x > 0.0` ergibt sich für `x = 10` der Wert `True`, für `x = None` der Wert `False` (der Vergleich `x > 0.0` wird wegen `and` nicht mehr ausgewertet) und für `x = "5"` ein `TypeError`, weil ein String nicht mit einer Zahl verglichen werden kann. Die umgeschriebene Form `x is not None and x > 0.0` ist für Menschen meist leichter zu lesen. Siehe auch ⟶ `05b_Verzweigung`, wo solche Ausdrücke als Bedingungen verwendet werden.

Quiz

Bibliographie